

Create specialized sub-agents to delegate tasks

Agent structure

An agent is a Markdown file placed in `.claude/agents/` with a YAML frontmatter. Claude Code discovers it automatically at session startup.

```
---
name: code-reviewer
description: Use to review code before any commit
model: sonnet
tools: Read, Grep, Glob
---
```

The file body contains instructions in Markdown: role, methodology, examples, constraints.

Frontmatter fields

Field	Required	Role
name	Yes	kebab-case identifier
description	Yes	Activation trigger (50-100 chars)
model	No	Aliases such as <code>sonnet</code> , <code>inherit</code> , or a supported model ID
tools	No	Whitelist of accessible tools
memory	No	Persistent memory scope
maxTurns	No	Agentic turn limit
permissionMode	No	Permission control
isolation	No	<code>worktree</code> for git isolation
color	No	CLI color for visual distinction

Without `model`, a subagent inherits the parent model.

`inherit` selects that behavior explicitly. Environment and organization overrides can still apply.

The tools field: isolation by default

Limiting `tools` to what the agent actually needs reduces the exposure surface and accidental errors. A code review agent only needs `Read`, `Grep`, `Glob`. No reason to give it

`Bash` OR `Write`.

Persistent memory (v2.1.33+)

```
memory: project # .claude/agent-memory/<name>/
memory: user    # ~/.claude/agent-memory/
<name>/
memory: local   # .claude/agent-memory-local/
<name>/
```

`project` memory can be committed for team use; review it first. `local` memory stays private to the machine. Enabling agent memory automatically adds `Read`, `Write` and `Edit`, so omit `memory` for a strictly read-only agent.

Agent vs slash command

Criterion	Agent	Slash Command
Specialty	Yes, specific domain	No, generic workflow
Own memory	Yes (v2.1.33+)	No
Isolated tools	Yes, whitelist	No
Invocation	Automatic or manual	Manual only
Format	Markdown + frontmatter	Markdown template

Create an agent when the task recurs regularly with stable context and tools. Create a command when it is a one-off workflow or a procedure to follow.

Concrete example: code review agent

```
---
name: code-reviewer
description: Use PROACTIVELY after every code
modification
  to verify quality, security, and conventions
model: sonnet
tools: Read, Grep, Glob
---
```

Code Reviewer

Systematically verify: OWASP security, test coverage, project convention compliance, and introduced technical debt.

The keyword `PROACTIVELY` in the description encourages Claude to invoke the agent without waiting for an explicit request.